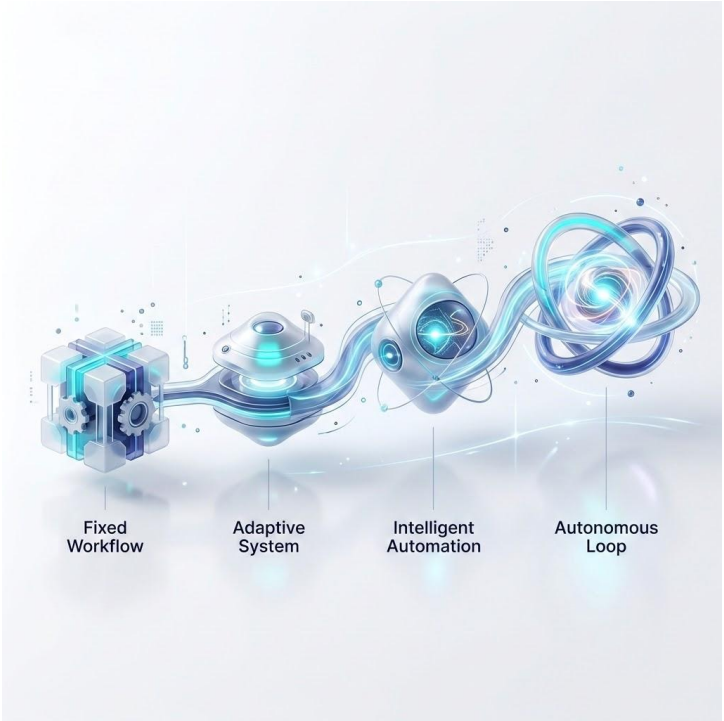
From RAG workflow to Agentic RAG



Agenda

01 **RAG Workflow**
The original idea of RAG

02 **Anatomy of RAG Workflow**
How is RAG workflow implemented under the hood

03 **RAG Evaluation**
Evaluation of RAG system

04 **Advanced RAG + Agentic RAG**
Advanced RAG techniques and how RAG is reshaped in agentic system

RAG Workflow

Conventional Workflow RAG: The "Retrieve-Then-Generate" Modular Blueprint

User Input Query

Linear Initiation

The journey begins when the user's query enters the conventional RAG system.

The query is immediately vectorized to prepare for the retrieval phase.

01

→

Retriever Module

Search & Extract

The query scans through the Vector DB to search for relevant information.

The output returns **Top-K Chunks** containing the closest context.

02

→

Prompt Augmentor

Context Injection

Combines the original **Query** + newly retrieved **Context Chunks** + **System Instructions** into a single prompt.

03

→

Generator / LLM

Passive Synthesis

The LLM receives the context-enriched prompt to explain and synthesize information.

Generates the **Final Answer** returned to the user.

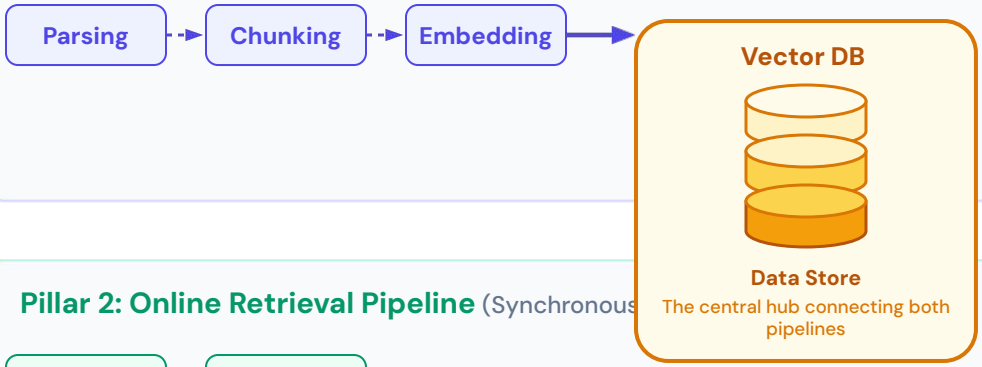
04

One-Way Static Module Chain: Search runs only once before LLM invocation • No two-way interactive feedback loop

RAG Workflow

The Two Main Pillars of a RAG System

Pillar 1: Offline Indexing Pipeline (Asynchronous, background processing)

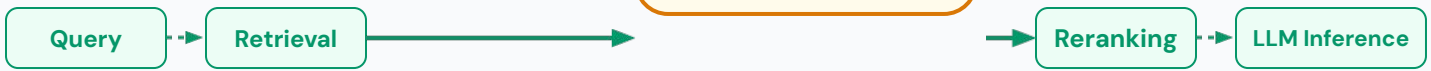


Data preparation & optimization:

The offline indexing pipeline runs in the background, processing asynchronously to prepare data (Parsing, Chunking, Embedding, Vector DB).

This pillar specifically **prioritizes throughput** and **data accuracy**.

Pillar 2: Online Retrieval Pipeline (Synchronous



Interactive requests:

Receives queries (Query) and performs rapid information retrieval (Retrieval).

Real-time service:

Serves users in real time. Requires optimizing for ultra-low latency (**under 1 second**) while ensuring the highest context accuracy.

Anatomy of RAG

Data Parsing & Extraction: Unifying All Formats into Structured Markdown

1. Diverse Raw Inputs

Scanned PDFs & Images
Multi-column layouts with embedded graphic schematics

Office Docs (DOCX / PPTX)
Hierarchical text blocks, notes, and visual slides

Financial Reports
Complex, nested multi-page tables and balance sheets

Scientific Papers
Academic layouts containing dense math LaTeX equations

2. Modern Parsing Engines

Docling (IBM)
Universal document layout converter supporting multi-format PDF/DOCX to JSON/MD

MinerU / Marker
Academic-focused parsers designed for complex LaTeX math and formula extraction

VLM Parser (GPT-4o/Claude)
Vision-Language Models serving as visual OCR pipelines to read raw document canvas

3. Clean Semantic Markdown

```
# Executive Summary
The primary RAG workflow benchmark.
## Key Metrics Table
| Q3 | Revenue | Growth |
|---|---|---|
| RAG-A | $1.2M | +14% |
## Formula Verification
$$ E = mc^2 \quad \text{and} \quad \sum_{i=1}^n x_i $$
## Document Schema
![System Diagram](fig1.png)
```

The "Lingua Franca" Philosophy: Why Structured Markdown Rules RAG

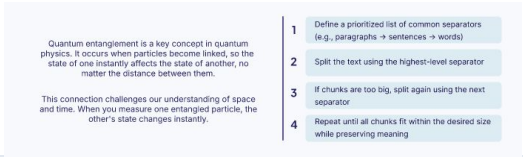
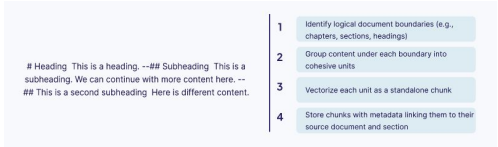
Naive OCR (Broken Semantics) • Incorrect
Drops layout rules. Multi-column texts merge blindly, tables break into random line fragments, mathematical equations distort into unreadable symbol errors, destroying context validity.

Structured Markdown (Layout-Aware) • Correct
Preserves precise hierarchy (# Headings), maintains high-fidelity nested tables, retains math formulas using LaTeX notation, conserves tokens and optimizes compatibility with LLMs.

Ref: https://docling.ai/
https://github.com/opendatalab/mineru
https://github.com/datalab-to/marker

Anatomy of RAG Workflow

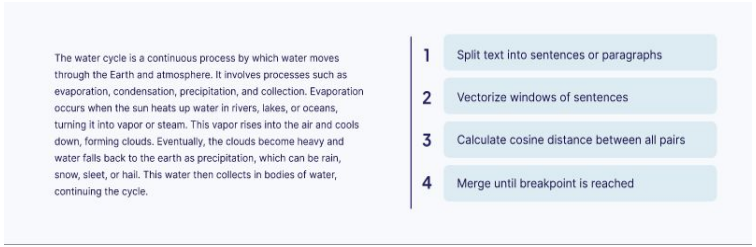
Text Chunking Strategies

Strategy	Example Chunks	What to Notice
Fixed-size + Overlap Strict character limit cuts with context window		**Cuts blindly** Overlap preserves boundary context but duplicates text
Recursive Hierarchical split-and-merge algorithm		**Dynamic fallback** Falls back to the largest separator that fits the size limit
Document-based Structure-aware boundaries		**Preserves meaning** Draws exact boundary boxes around headers with content

Ref:
https://weaviate.io/blog/chunking-strategies-for-rag
https://qdrant.tech/course/essentials/day-1/chunking-strategies/#text-chunking-strategy-comparison

Anatomy of RAG Workflow

Advanced Chunking Strategies

Strategy	How it works / Visual	Trade-off
Semantic chunking Embed sentences detect topic shifts via similarity drop split		**Better boundaries** Requires embedding model run at chunking time

Late chunking

Embed full document first with long-context model, then chunk & pool

Requires specialized long-context embedding models

Hierarchical

Multi-layered tree structuring (Summary Section Paragraph)

Multi-granularity retrieval; complex parent tracking

LLM-based chunking

psychology

LLM summarizes or splits sections into clean semantic propositions

Highest quality; expensive & slow

Agentic chunking

smart_toy

Intelligent agent analyzes document format and determines best mix dynamically

Most flexible; highly complex system design

Ref:
https://weaviate.io/blog/chunking-strategies-for-rag#semantic-chunking-context-aware-chunking
https://www.firecrawl.dev/blog/best-chunking-strategies-rag

Anatomy of RAG Workflow

Optimizing Vector Search at Scale

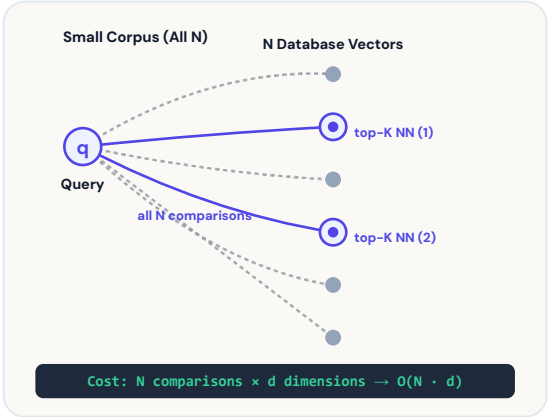
Challenges & Solutions for Large-Scale Search

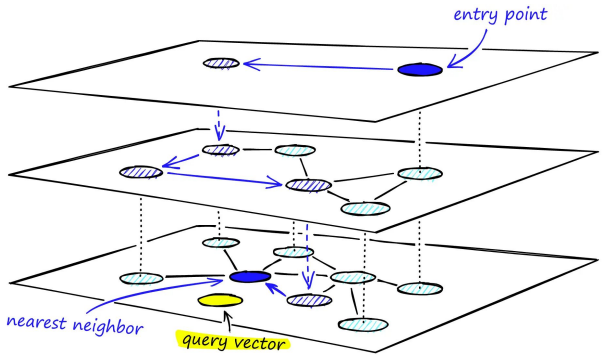
Exact KNN Bottleneck (O(N * d)):
As data scales to millions of vectors, Exact KNN becomes too slow, failing to meet real-time production requirements.

ANN & HNSW Solutions:
Using Approximate Nearest Neighbors (ANN) with the HNSW algorithm achieves an excellent balance between Recall and Latency.

Vector Compression with IVF-PQ:
To optimize memory, the IVF-PQ (Inverted File with Product Quantization) solution compresses vectors highly efficiently.

Flexible Metadata Filtering:
Utilizing Pre-filtering, Post-filtering, or Single-stage Payload Filtering strategies to maintain optimal query speed.





Ref:
https://www.pinecone.io/learn/series/faiss/hnsw/
https://milvus.io/learn-milvus/hnsw
https://cfu288.com/blog/2024-05_visual-guide-to-hnsw/

Anatomy of RAG Workflow

Why use a Vector Database?

Search Library vs. Database

01

Search Library (e.g. FAISS, HNSWlib): Provides raw, ultra-fast similarity search inside an in-memory process, but lacks production systems engineering.

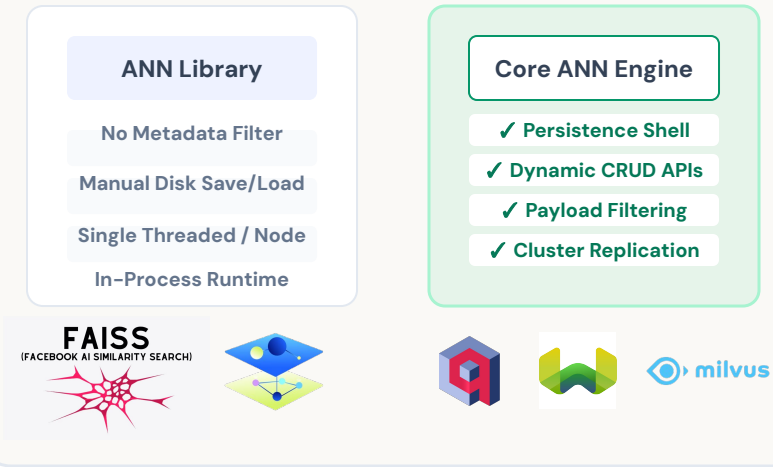
Production Requirements:

- **Persistence:** Vectors persist across process restarts.
- **Dynamic CRUD:** Modify documents without rebuilding indices.
- **Metadata Filtering:** Combine similarity with hard constraints.
- **Scalability:** Live replication and clustering across nodes.

Vector Database (e.g. Qdrant, Weaviate, Milvus): Wraps indexing with storage, APIs, multi-tenant security, clustering, and rich observability.

Infrastructure Architecture

02



Ref:
https://qdrant.tech/articles/what-is-a-vector-database/

Anatomy of RAG Workflow

Modern SOTA Reranking: Listwise Attention & Jina Reranker v3/v3.5

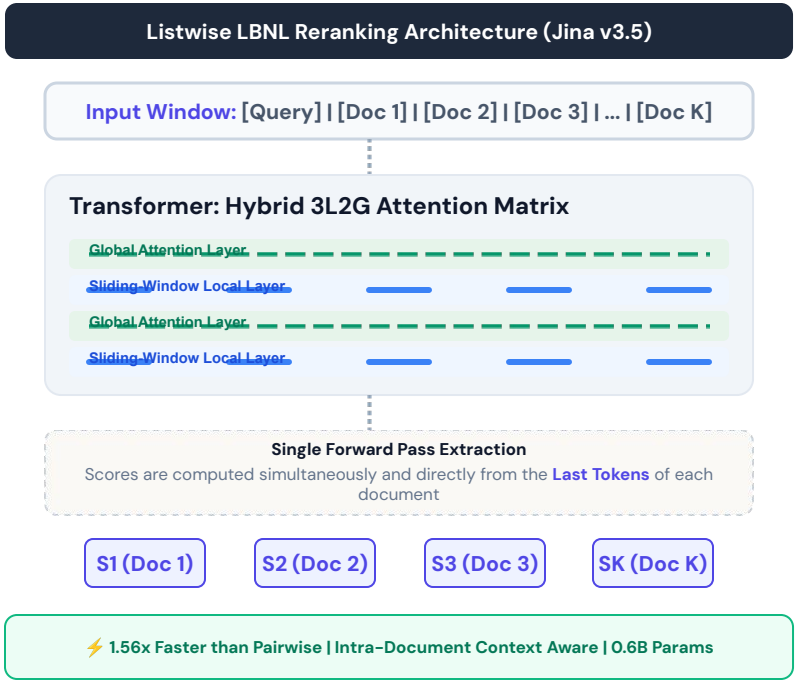
Why is Pairwise Cross-Encoder Imperfect?

Scoring each **[Query, Doc]** pair independently completely ignores the cross-correlation context between documents. In practice, **Document A** may contain foundational context that directly supports a better understanding of **Document B**.

Listwise Reranking (LBNL Architecture)

A breakthrough based on the **Last-but-Not-Late** philosophy that fully leverages the power of Transformers in a single pass:

- Input the entire sequence **[Query + Doc 1 + ... + Doc K]** into a single context window of the model.
- Compute cross-correlation in just **1 single forward pass** instead of repeating it K separate times like traditional Cross-Encoders.
- Extract relevance scores directly from the **Last tokens** representing each document.



RAG Evaluation

Deterministic RAG Evaluation: Retrieval Ranking & Generation Metrics

Nature of Deterministic Metrics: Mathematical, quantitative, 100% reproducible metrics, $0 API cost, measuring Retrieval & Generation layers independently.

1. Retrieval Layer Metrics (Ranking & Search)

HIT RATE @ 10

94.0%

✓ Target (>90%) met

MRR @ 10

0.82

✓ Ideal first-chunk rank

NDCG @ 10

0.88

✓ Highly optimized sequence

MAP @ K & Precision @ K: Ratio of truly useful chunks out of total retrieved chunks.

Measures noise and specificity of the information filter.

2. Generation Metrics (Exact & Semantics)

EXACT MATCH (EM)

86.2%

Short facts/structured

ROUGE-L & BLEU:
Character-sequence evaluation.
Levenshtein distance for physical text deviation.

3. System Telemetry & Embedding Drift

P95 LATENCY & COST

450ms / $0.0012

⚠️ Latency spikes checked

Evidently AI Drift: Monitors distribution shift between vector indexing vs query.

Schema Validation & PII Regex check pass.

CI/CD AUTOMATED REGRESSION TEST GATE PASSED ✓

Ref:
https://deepeval.com/guides/guides-rag-triad

RAG Evaluation

LLM-as-a-Judge: Qualitative Evaluation & The RAG Triad

Why LLM-as-a-Judge?

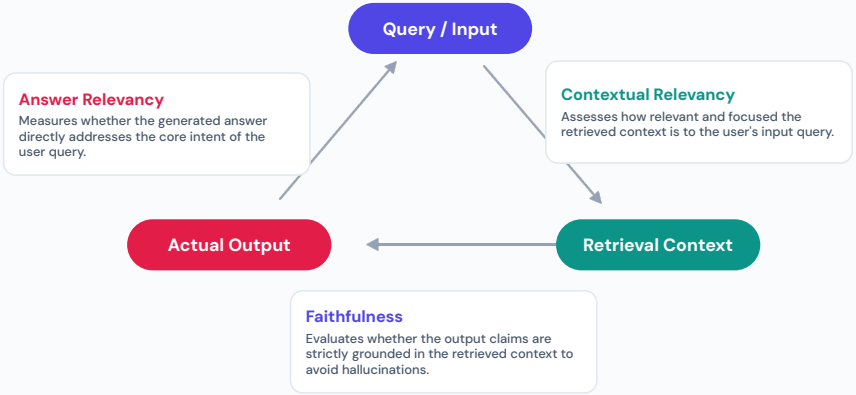
Quantitative metrics like **ROUGE & BLEU fall short** when evaluating complex semantic alignment and nuanced outputs.

Instead, a highly capable LLM (such as **GPT-4o or Claude 3.5 Sonnet**) acts as an expert "Judge" to qualitatively grade answers based on human-like standards.

Standard Evaluation Frameworks

Using industry-standard toolkits like **DeepEval (Confident AI)** and **RAGAS** to systemize qualitative tracking:

- **G-Eval Framework:** Flexible evaluation leveraging custom, structured rubrics tailored to specific enterprise domain rules.
- **Synthetic Test Data:** Automatically generates diverse, high-coverage testing datasets to robustly stress-test RAG pipelines without requiring manual human labeling.



Ref:
https://deepeval.com/guides/guides-rag-triad

Limitations

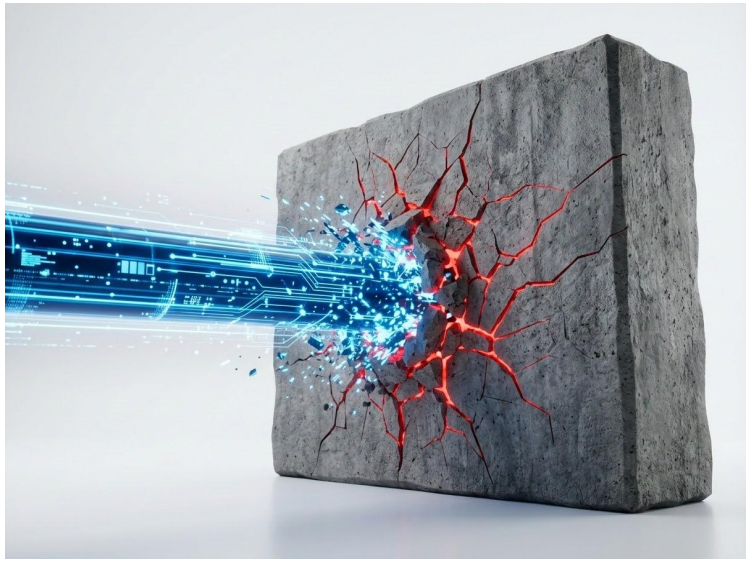
RAG Workflow Fails Against 4 Key Limitations

01. Structural Loss
Vector distance ignores the hierarchical context of documents.

02. Multi-Hop Reasoning Failure
One-way pipelines cannot chain logic across multiple documents.

03. Rigid Query Execution
Searches only once without knowing how to self-adjust.

04. Hallucination do Noise
Optimizing Top-K still accidentally introduces noise into the prompt.

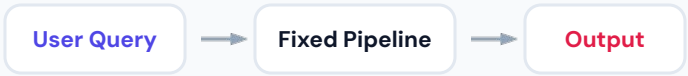


Agentic RAG

The Shift to Agentic RAG

Workflow RAG

Fixed Pipeline Model



- Queries pass through a **fixed pipeline** to generate results.
- Retrieval acts as a **static "framework"**, lacking dynamic flexibility.
- Lacks the ability to self-correct or perform intermediate evaluations.

Agentic RAG

Active Agent Brain

- Queries go through the controller brain for Planning, Tool Selection & Execution.
- Retrieval becomes a **flexible "tool"** actively invoked by the Agent when needed.
- The system actively **evaluates** and **self-corrects** before returning results.

Core Shift: *Retrieval is no longer a fixed "framework," but a flexible "tool" actively controlled by the Agent.*

Agentic RAG

Control Flows in Agentic RAG

01

Query Decomposition

Allows the Agent to autonomously decompose complex questions into **sub-queries** for optimal multi-hop retrieval.

02

Dynamic Routing

Helps the Agent autonomously and flexibly select between optimal resources such as **Vector DB**, **SQL**, or **Web Search**.

04

Self-RAG & Reflection

Uses **Reflection Tokens** for the Agent to self-check for hallucinations and accurately adjust answers in real-time.

03

Corrective RAG (CRAG)

Automatically evaluates retrieval quality and actively redirects to **Web Search** if low-quality internal data is detected.

Agentic RAG

ReAct Loop – The Heart of Agentic RAG

react_loop.py

```python
# Initialize while loop to optimize search
while not agent.confident():

        # 1. Reason & Plan
        thought = agent.think()

        # 2. Select appropriate tool
        tool = agent.select_tool()

        # 3. Trigger query & Retrieve
        data = agent.execute_tool(tool)

        # 4. Memorize & Evaluate results
        agent.update_memory(data)

# Confidently synthesize the final answer
return agent.synthesize_answer()
```

1. Think & Plan

The **`think()`** step analyzes the context to self-direct and build a reasoning plan.

2. Select Tool

The **`select_tool()`** step actively selects the optimal search method.

`while`

4. Memorize

The **`update_memory()`** step accumulates, memorizes results, and observes to prepare for the next step.

3. Execute

The **`execute_tool()`** step triggers retrieval and collects raw data.

** The continuous loop completely eliminates the one-way flow, granting absolute autonomy to the Agent.*

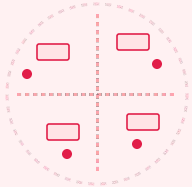
Ref: https://docs.langchain.com/oss/python/deepagents/overview

Advanced RAG

Natural Book-Reading Intuition Instead of Vector Chunking

Limitation: Fixed 512-Token Chunking

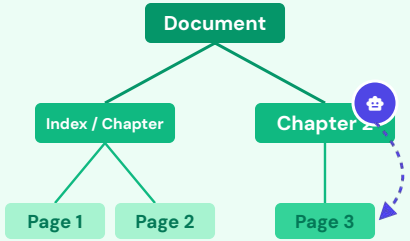
Traditional methods chunk documents into fixed **512-token** vectors, disrupting the natural context structure and connections between pages.



Discrete Vector Space

Solution: PageIndex Tree Structure

Builds a hierarchical tree structure from **Document to Page Node**. Approaches data with a natural reading intuition: browse TOC, select chapter, select page, then read closely.



Hierarchical Tree Map (PageIndex Map)

Reasoning-Based Retrieval

1. LLM as an Agent

The LLM actively reads the Hierarchical Tree Map to reason and precisely locate the branch containing the target information.

2. Direct Jump Retrieval

The system jumps directly to the target Page Node to extract data without tedious linear scanning.

3. Eliminating Vector Databases

A flawless and precise extraction process that completely eliminates complex dependencies on traditional Vector databases.

Ref:
https://pageindex.ai/

Production Optimization

Optimizing Agentic RAG in Production

Prompt & Context Caching

-90%
Cost & Latency

Prompt Caching is a breakthrough technique that efficiently stores and reuses the Agent's repetitive context.

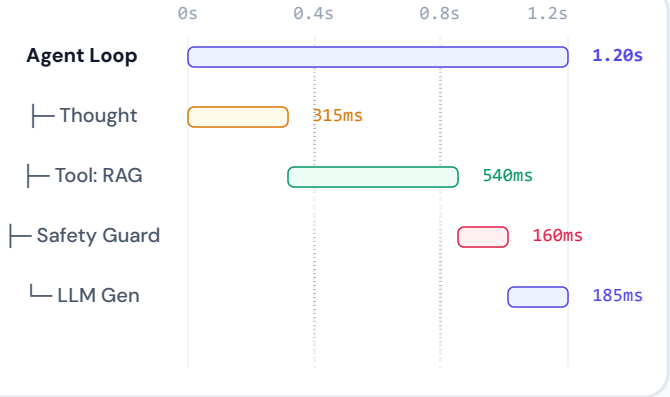
Context Compression & Pruning

Compress prompts before entering the Agent Loop to maximize resource savings and avoid context drift during sessions.

Raw Context ⇢ Compress & Prune ⇢ Agent Loop

Agent Observability & Telemetry

Track detailed reasoning steps, Tool Calls, and evaluate safety via monitoring systems like **LangSmith** or **TraceAI**.



Finalize

Workflow RAG vs. Agentic RAG: Decision Matrix

Workflow RAG

Optimized Performance, Speed & Cost

High Speed & Low Cost
Fast response times, optimized API call costs.

Simple Queries
Best suited for direct information retrieval queries.

Focus on Parsing & Reranking
Requires precise data preprocessing and reranking.

Agentic RAG

Ideal for Complex Logic & Reasoning Chains

Higher Latency & Cost
Due to iterative reasoning loops and multiple LLM calls.

Complex Problem Solving
Connects multi-hop logic across diverse data sources.

Self-Correction Capabilities
Automated self-evaluation, feedback, and refinement.

Selection Rule: Fast & Simple ➔ **Workflow RAG** | Complex & Multi-hop ➔ **Agentic RAG**